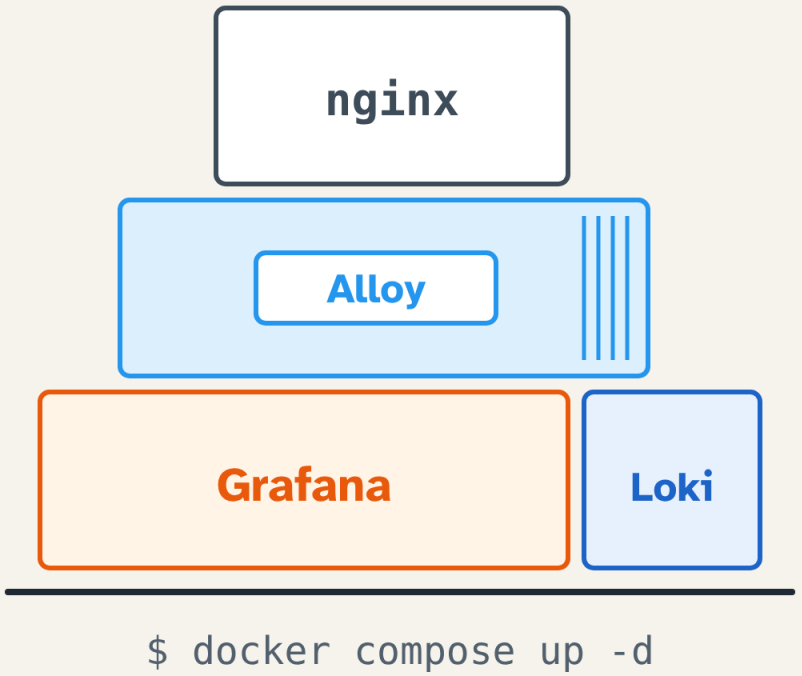
LEZIONE 3

Docker e Docker Compose

ITS CyberSecurity - Jesi (AN) 2026/2027

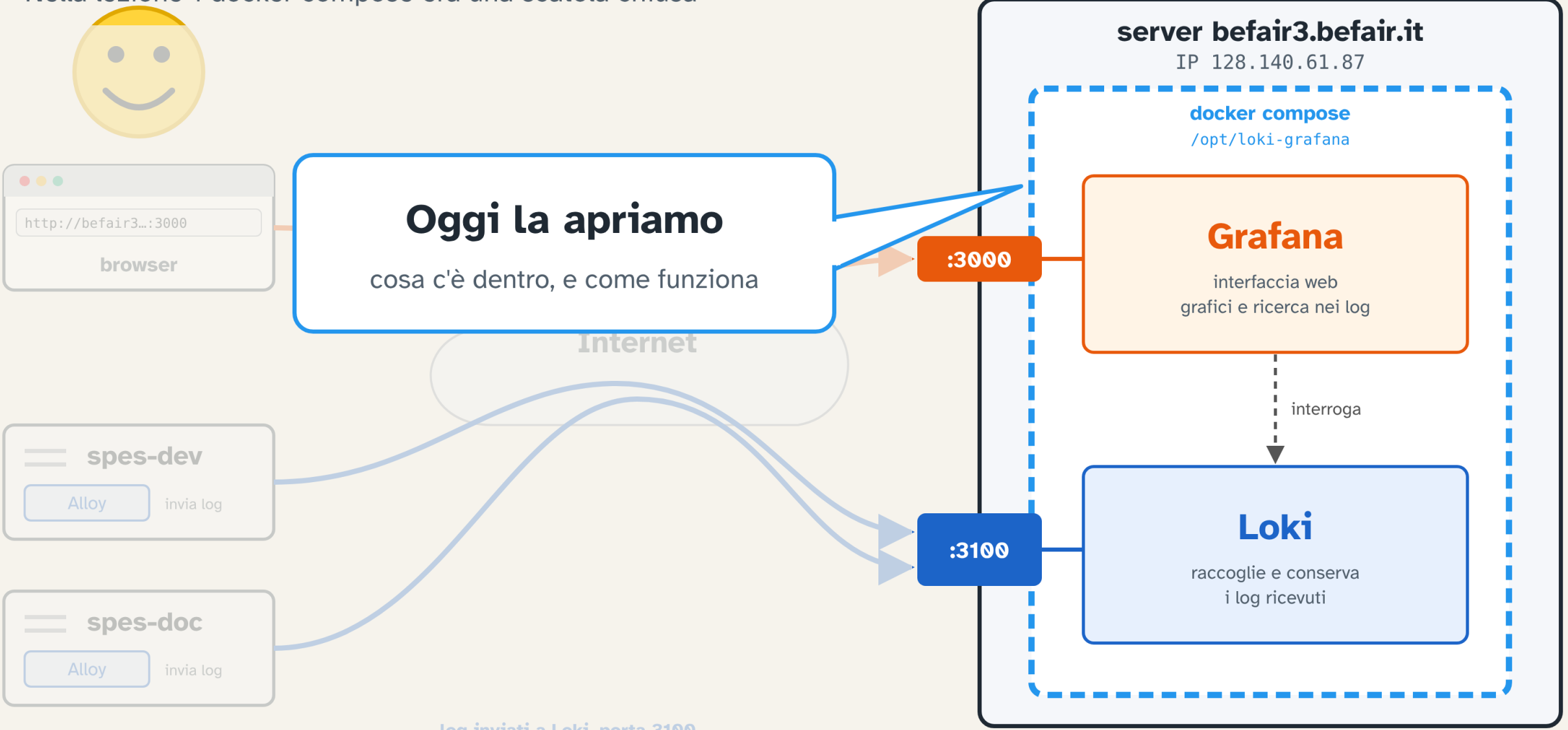
Corso Logging, Monitoring, Troubleshooting

prof. Luca Ferroni



Dove eravamo

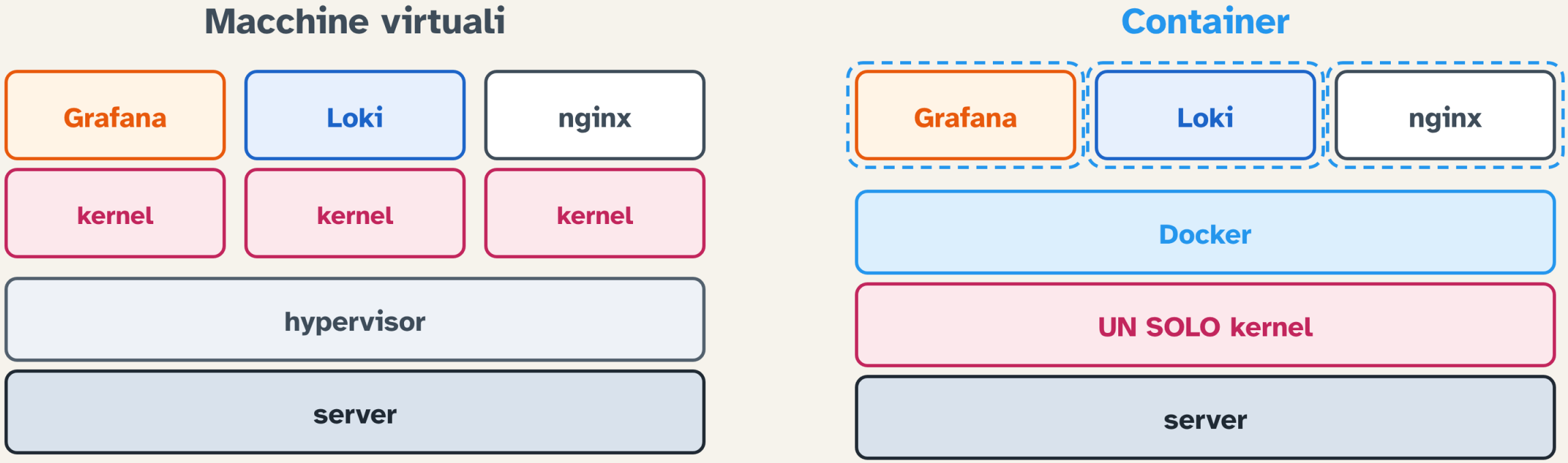
Nella lezione 1 docker compose era una scatola chiusa



log inviati a Loki, porta 3100

Un container non è una macchina virtuale

I container condividono il kernel dell'host (lezione 2)

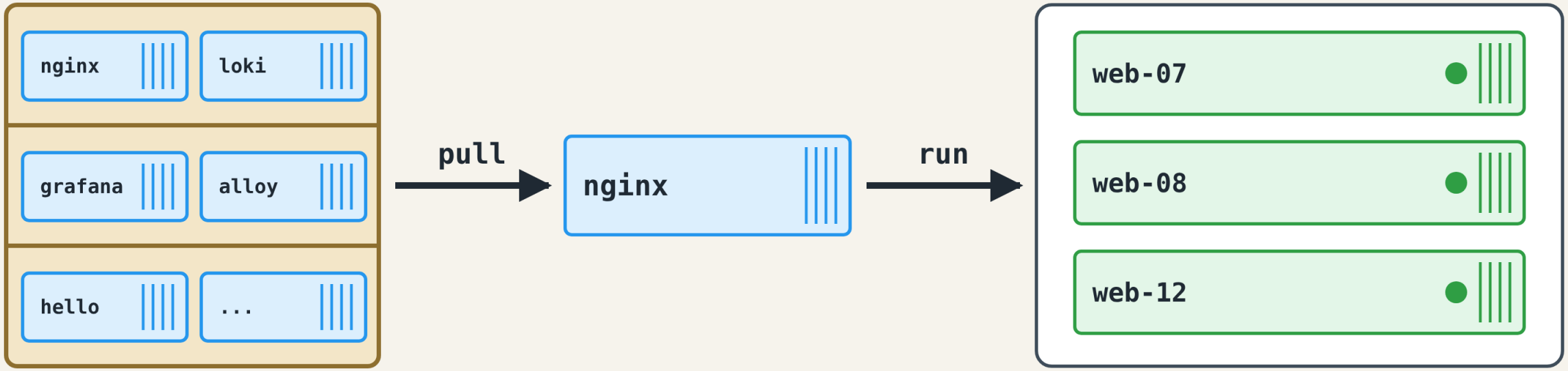


Un container è un processo dell'host, chiuso in una scatola.

Parte in un secondo. Lo vedi con ps aux, come il miner della lezione 2.

Registry, immagine, container

Ridisegnato da Microsoft Learn, "Docker overview"



Registry

il magazzino

Immagine

il pacchetto fermo

Container

l'immagine in moto

Da una sola immagine si avviano tanti container.

Come si legge il nome di un'immagine

Il nome dice da dove viene e quale versione è

```
grafana/loki:3.7.0
```

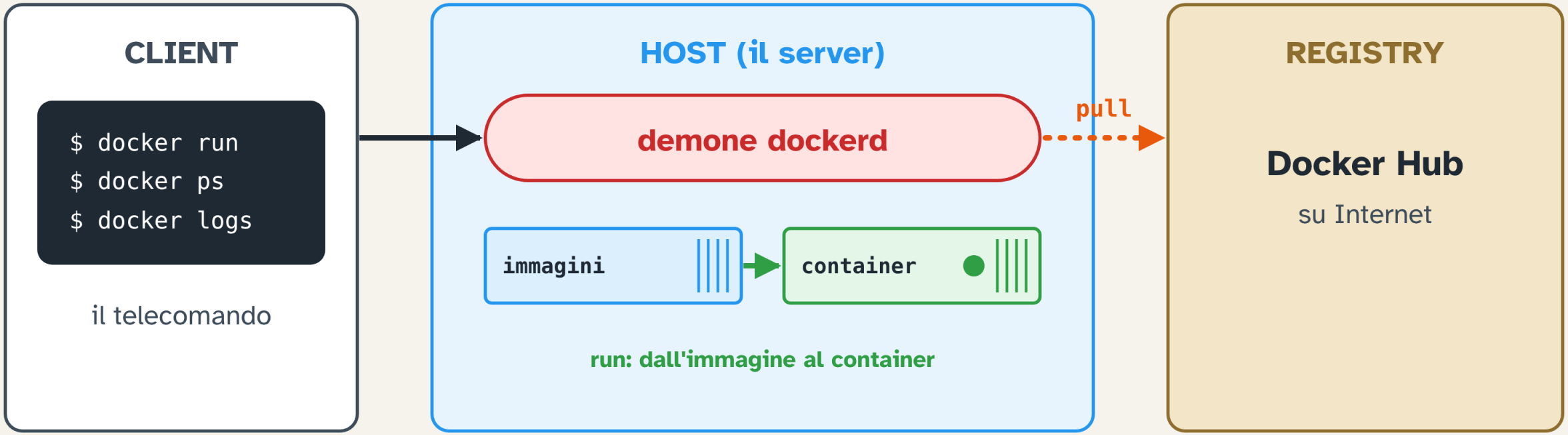
chi la pubblica — **il nome** — **la versione (tag)**

Senza tag Docker prende :latest

che oggi è una versione e domani un'altra

Chi fa cosa

Ridisegnato da Nordic APIs, via devopedia.org



Il comando docker chiede, il demone fa.

Chi può usare docker è, di fatto, root sulla macchina.

Torniamo al centro commerciale

Le parole di Docker nella metafora della lezione 2

`registry`	l'elenco dei progetti di negozio apribili
`immagine`	il progetto di un negozio
`container`	un negozio aperto
`-p 8007:80`	porta 8007 in facciata, bancone 80 dentro
`dockerd`	la direzione del centro
`docker logs pippo`	il registro di cassa del negozio pippo

Il primo container

L'output racconta da solo la slide di prima

```
$ docker run hello-world
Hello from Docker!
...
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
 3. The Docker daemon created a new container from that image ...
 4. The Docker daemon streamed that output to the Docker client ...
```

1. il client chiede al demone
2. il demone scarica l'immagine
3. crea e avvia il container
4. l'output torna a te

Dov'è finito?

docker ps mostra solo i container accesi

```
$ docker ps
CONTAINER ID   IMAGE   STATUS

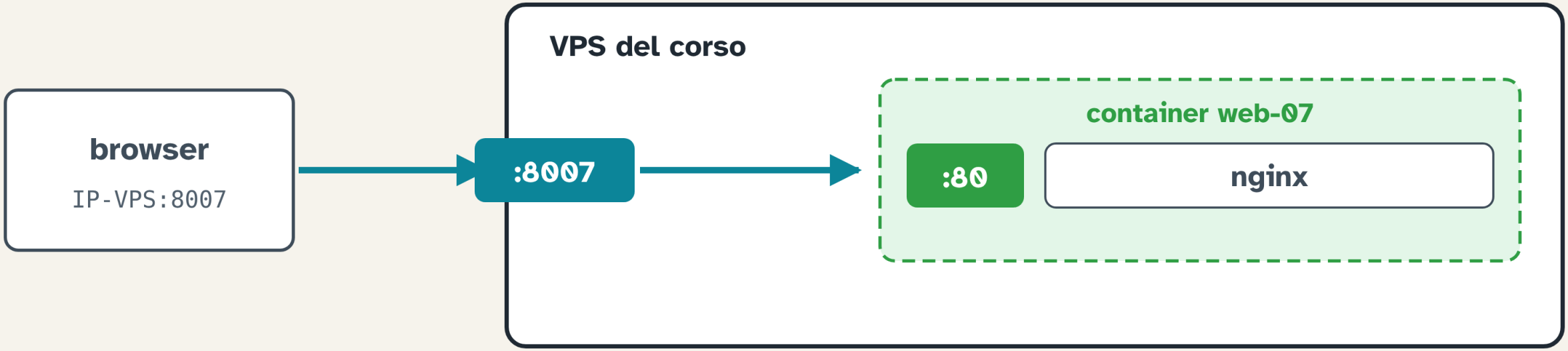
$ docker ps -a
CONTAINER ID   IMAGE         STATUS
4c1d2e9f0a1b   hello-world   Exited (0) 10 seconds ago
```

Un container vive quanto il suo processo.

hello-world stampa e termina: il container resta lì, fermo.

Un negozio per ciascuno

-p PORTA_FUORI:PORTA_DENTRO




```
$ docker run -d --name web-07 -p 8007:80 nginx:alpine
```

Fuori 8000 + il tuo numero. Dentro, nginx ascolta sempre sulla 80.

Funziona?

Controlliamo dal server, poi dal browser

```
$ docker ps
NAMES    STATUS         PORTS
web-07   Up 5 seconds   0.0.0.0:8007->80/tcp

$ curl -I http://localhost:8007
HTTP/1.1 200 OK
Server: nginx
```

Poi dal browser: http://IP-del-VPS:8007

Due cose da sapere sulle porte

Porta già occupata

```
$ docker run ... -p 8007:80 ...
Error ... Bind for 0.0.0.0:8007
failed: port is already allocated
```

La porta in facciata è una sola.
Se la prende un compagno, il tuo
run fallisce. Leggi l'errore.

È su Internet

Una porta pubblicata con -p è
aperta a tutti, anche se il
firewall ufw dice di no.

Solo dal server stesso:

`-p 127.0.0.1:8007:80`

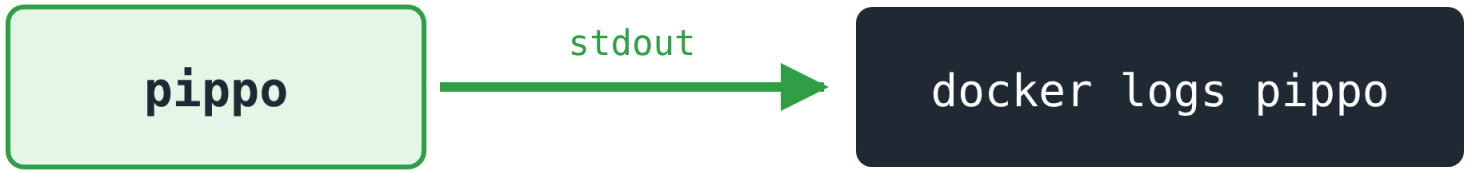
A fine lezione:

`docker rm -f web-07`

Dove finiscono i log

Due strade diverse

Il container



Docker raccoglie
ciò che stampa

Il demone



è un servizio di
systemd, come sshd

Con compose: docker compose logs -f grafana (lezione 1)

I log di nginx

Ogni ricarica del browser è una riga

```
$ docker logs web-07
203.0.113.25 - - [15/Oct/2026:09:12:44 +0000] "GET / HTTP/1.1" 200 615 ...
203.0.113.25 - - [15/Oct/2026:09:12:51 +0000] "GET /pippo HTTP/1.1" 404 ...

$ docker logs web-07 2>/dev/null | awk '{print $1}' | sort | uniq -c
     14 203.0.113.25
      3 198.51.100.7
```

Quasi il formato di apache_access.log: valgono i comandi della lezione 2.

L'ora è in UTC (+0000): 09:12 UTC sono le 11:12 in Italia.

Il sito non risponde

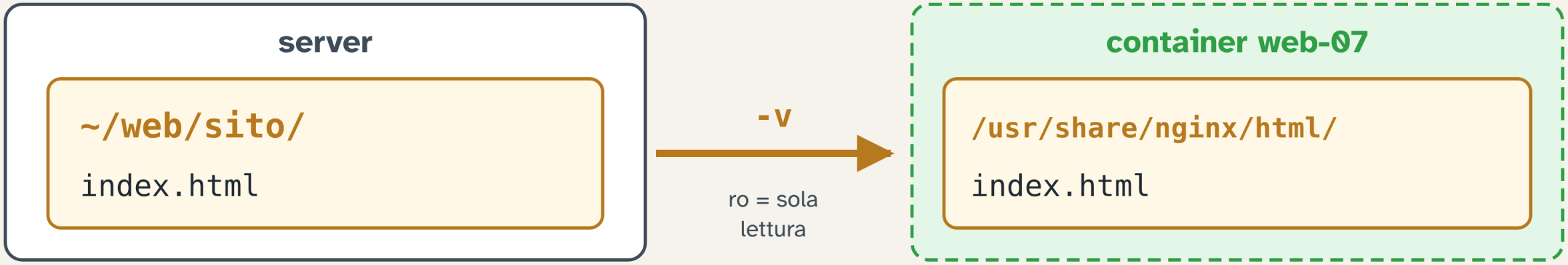
Un livello alla volta, come nelle lezioni 1 e 2

1. `nc -zv IP-VPS 8007` — dal PC: la porta risponde?
2. `ss -ltnp | grep 8007` — sul server: chi ascolta?
3. `docker ps -a` — il container gira?
4. `docker logs web-07` — cosa ha detto prima di fermarsi?

Volumi: i dati stanno fuori

Lo stesso file, non una copia: il container si butta, i dati restano

```
$ mkdir -p ~/web/sito
$ echo '<h1>ciao</h1>' > ~/web/sito/index.html
```




```
$ docker rm -f web-07
$ docker run -d --name web-07 -p 8007:80 \
    -v ~/web/sito:/usr/share/nginx/html:ro nginx:alpine
```

Ricarica http://IP-del-VPS:8007 e compare: ciao

Entrare nel container

docker exec avvia un comando dentro un container acceso

```
$ docker exec -it web-07 sh
/ # ls /usr/share/nginx/html
index.html
/ # cat /etc/os-release
NAME="Alpine Linux"
/ # exit
```

Dentro c'è un altro Linux, piccolo: Alpine.

Come fare ssh in un altro server, ma senza rete.

Da docker run a compose.yml

```
$ docker run -d --name web-07 -p 8007:80 \
    -v ~/web/sito:/usr/share/nginx/html:ro nginx:alpine
```

~/web/compose.yml

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8007:80"
    volumes:
      - ./sito:/usr/share/nginx/html:ro
    restart: unless-stopped
```

Stesso container, scritto in un file.

Si legge, si copia, si versiona con git.

YAML: spazi, mai tab.

I comandi di docker compose

`docker compose up -d`	avvia tutto
`docker compose ps`	chi gira?
`docker compose logs -f web`	log in tempo reale
`docker compose stop`	ferma, senza cancellare
`docker compose down`	ferma e CANCELLA i container

Sempre dalla cartella del compose.yml, come /opt/loki-grafana nella lezione 1.

Riapriamo il caso della lezione 1

Perché Grafana non era ripartito da solo?

`/opt/loki-grafana/compose.yml (estratto)`

```
services:
  loki:
    image: grafana/loki:3.7.0
    ports: ["3100:3100"]
    # restart: ???
  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
    # restart: ???
```

03:30 UTC
Grafana e Loki si spengono

dopo
nessuno li riaccende

mattina
Grafana irraggiungibile

Senza restart: un container fermo resta fermo.

Chi li ha spenti alle 03:30? Lo chiediamo a journalctl -u docker.

La cura: una riga per servizio

```
restart: unless-stopped
```

Riparte da solo se si ferma o se il server si riavvia.

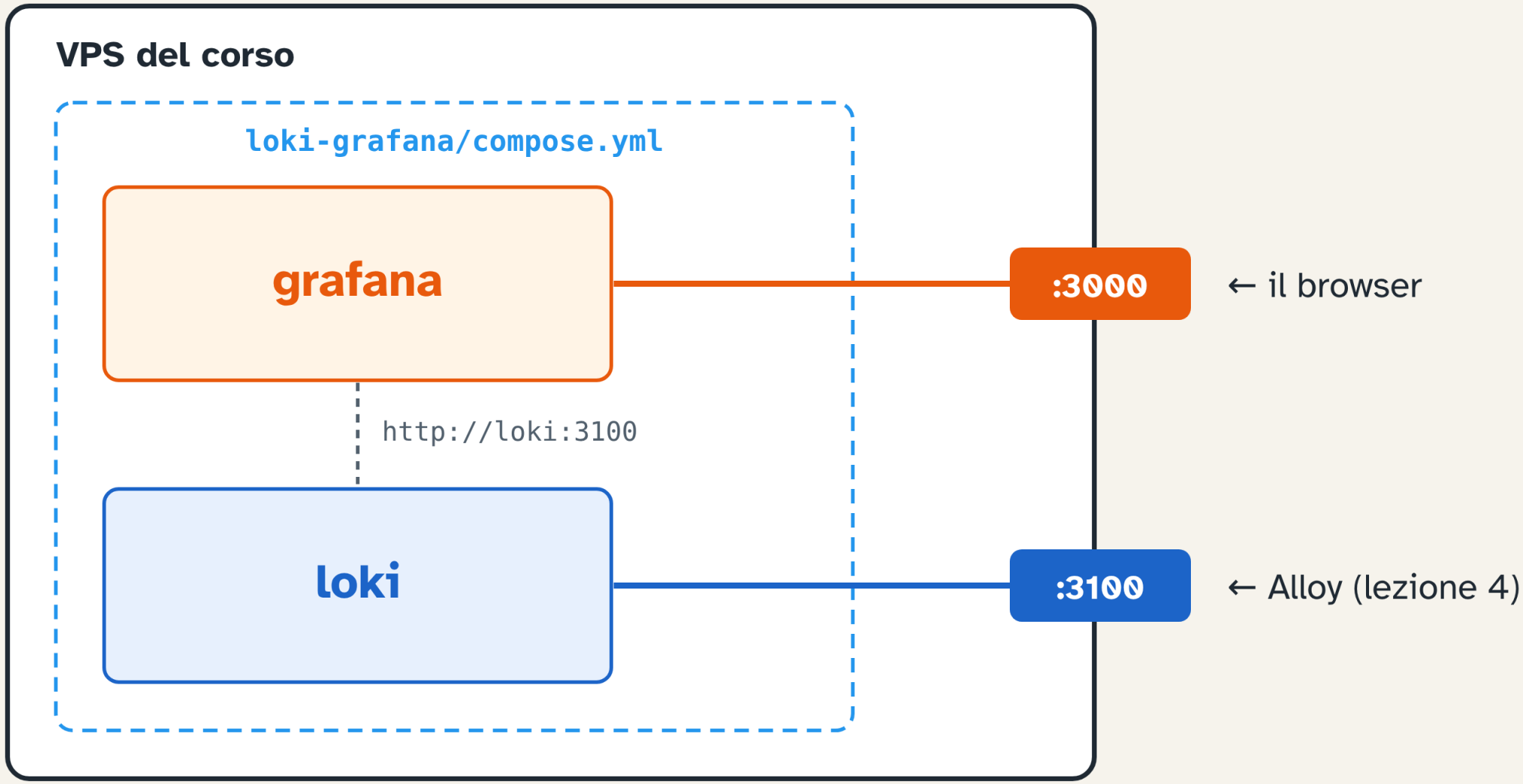
Resta fermo solo se l'hai fermato tu.

Il compose di Alloy ce l'ha già.

```
$ sudo systemctl restart docker
$ docker compose ps    # prima senza, poi con
```

Lo stack del corso

Lo stesso compose di befair3, su un altro server



Lo facciamo insieme

1. `cd its-jesi-lessons/logging/loki-grafana`
2. `docker compose up -d`
3. `docker compose ps` — entrambi Up?
4. `curl http://IP-VPS:3100/ready` — risposta: ready
5. `browser: http://IP-VPS:3000` — admin / admin
6. `sorgente dati Loki: http://loki:3100`

Prima di uscire

1. Immagine e container: che differenza c'è?
2. Dove leggo i log di un container?
3. E quelli del demone Docker?
4. A cosa serve restart: unless-stopped?

Per la lezione 4

Sul vostro PC funziona docker run hello-world?

Annotate se usate Linux, WSL2 o Docker Desktop.